Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «АПО ЭВМиС»
Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – обработка символьных данных»

Выполнил:
Студент 4-го курса
Группы АС-66
Кацевич А.Ю.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, практическая работа с подсистемой регистровой и оперативной памяти.

Ход работы

Вариант 6

Проверяемое условие 6: текст содержит равное количество прописных латинских и прописных русских букв.

Правило преобразования 6: заменить каждую строчную латинскую букву соответствующей прописной буквой, а прописную – строчной.

Постановка задачи: дан текст – непустая последовательность не длиннее ста символов, признак конца ввода – точка (в текст не входит). Проверить, удовлетворяет ли текст условию. Если условие выполнено, преобразовать текст по правилу, иначе выдать сообщение. Преобразованный текст напечатать. Ввод, проверка, обработка и печать выполняются последовательно, отдельными частями программы.

Платформа. Как и в лабораторной работе №1, программа сделана для Mac с процессором Apple (ARM64): процедуры написаны на ассемблере ARM64 в файле text_proc.S, а ввод, вывод и самопроверка – в файле lab2.cpp.

Текст хранится в кодировке Windows-1251, один символ занимает один байт: «A»–«Z» = 0x41–0x5A, «А»–«Я» = 0xC0–0xDF, «Ё» = 0xA8. Ввод из терминала (UTF-8) переводится в Windows-1251 функцией iconv и обратно при печати. Буква «Ё» считается русской прописной. Если прописных букв нет ни одного вида (0 = 0), условие считается выполненным.

Процедуры на ассемблере: text_is_valid – проверка, что в тексте нет управляющих символов (иначе последовательность не является текстом и преобразовываться не будет); check_equal_upper – проверка условия; swap_latin_case – преобразование: регистр латинской буквы меняется инверсией бита 0x20. Дополнительная память, зависящая от длины текста, и глобальные переменные не используются. В варианте 6 нет пометки «с.к.», поэтому строковые команды с префиксами повторения не обязательны.

Текст программы (text_proc.S)

```
// Лабораторная работа №2, вариант 6. Процедуры обработки текста на ассемблере.
// Платформа: Mac с процессором Apple (ARM64), ассемблер Clang.
//
// Текст хранится в кодировке Windows-1251, один символ = один байт:
//   'A'..'Z'  = 0x41..0x5A     'a'..'z' = 0x61..0x7A
//   'А'..'Я'  = 0xC0..0xDF     'Ё'      = 0xA8
//
// Во всех процедурах: x0 = адрес первого символа, x1 = длина текста в байтах,
// результат (если есть) возвращается в w0. Дополнительная память не используется:
// всё хранится в регистрах. Глобальных переменных нет.
```

```
    .text
    .align 2

// ---------------------------------------------------------------
// int text_is_valid(const unsigned char* text, long len)
// Возвращает 1, если в тексте нет управляющих символов (код < 0x20 и 0x7F).
// Иначе возвращает 0: последовательность не является текстом.
// ---------------------------------------------------------------
    .globl _text_is_valid
_text_is_valid:
    cbz  x1, Lvalid_yes          // пустой текст нечего проверять
Lvalid_loop:
    ldrb w2, [x0], #1            // w2 = очередной символ, адрес сдвигается на 1
    cmp  w2, #0x20
    b.lo Lvalid_no               // код меньше 0x20 - управляющий символ
    cmp  w2, #0x7F
    b.eq Lvalid_no               // код 0x7F (DEL) - тоже управляющий
    subs x1, x1, #1              // осталось символов на один меньше
    b.ne Lvalid_loop
Lvalid_yes:
    mov  w0, #1
    ret
Lvalid_no:
    mov  w0, #0
    ret

// ---------------------------------------------------------------
// int check_equal_upper(const unsigned char* text, long len)
// Условие варианта 6: количество прописных латинских букв равно
// количеству прописных русских букв. Возвращает 1 (верно) или 0 (неверно).
// w2 - счётчик латинских, w3 - счётчик русских.
// ---------------------------------------------------------------
    .globl _check_equal_upper
_check_equal_upper:
    mov  w2, #0
    mov  w3, #0
    cbz  x1, Lcheck_result
Lcheck_loop:
    ldrb w4, [x0], #1            // w4 = очередной символ
    sub  w5, w4, #0x41           // w5 = код - код 'A'
    cmp  w5, #25                 // беззнаковое сравнение: 0..25 значит 'A'..'Z'
    b.ls Lcheck_latin
    sub  w5, w4, #0xC0           // w5 = код - код 'А'
    cmp  w5, #31                 // 0..31 значит 'А'..'Я'
    b.ls Lcheck_russian
    cmp  w4, #0xA8               // отдельно проверяем 'Ё'
    b.ne Lcheck_next
Lcheck_russian:
    add  w3, w3, #1
    b    Lcheck_next
Lcheck_latin:
    add  w2, w2, #1
Lcheck_next:
    subs x1, x1, #1
    b.ne Lcheck_loop
Lcheck_result:
    cmp  w2, w3
    cset w0, eq                  // w0 = 1, если счётчики равны, иначе 0
    ret

// ---------------------------------------------------------------
// void swap_latin_case(unsigned char* text, long len)
// Правило варианта 6: строчную латинскую букву заменить прописной,
// а прописную - строчной. Остальные символы не меняются.
// Регистр латинской буквы переключается инверсией бита 5 (число 0x20).
// ---------------------------------------------------------------
```

```
    .globl _swap_latin_case
_swap_latin_case:
    cbz  x1, Lswap_done
Lswap_loop:
    ldrb w2, [x0]              // w2 = символ (адрес пока не сдвигаем)
    orr  w3, w2, #0x20         // w3 = тот же символ, приведённый к строчной
    sub  w3, w3, #0x61         // w3 = код - код 'a'
    cmp  w3, #25               // 0..25 значит, что это латинская буква
    b.hi Lswap_next
    eor  w2, w2, #0x20         // меняем регистр буквы
    strb w2, [x0]              // записываем обратно в текст
Lswap_next:
    add  x0, x0, #1
    subs x1, x1, #1
    b.ne Lswap_loop
Lswap_done:
    ret
```

Текст программы (lab2.cpp)

```
// Лабораторная работа №2, вариант 6.
// Условие: в тексте поровну прописных латинских и прописных русских букв.
// Правило: если условие выполнено, каждую латинскую букву заменить на букву
//          другого регистра (строчную на прописную, прописную на строчную).
// Ввод: непустой текст не длиннее 100 символов, признак конца - точка (в текст не входит).
//
// Проверка условия и преобразование написаны на ассемблере (файл text_proc.S).
// Этот файл отвечает за ввод, вывод и самопроверку.

#include <iconv.h>

#include <cstdlib>
#include <iostream>
#include <string>

using namespace std;

// ---------- Процедуры на ассемблере (text_proc.S) ----------
extern "C" {
int text_is_valid(const unsigned char* text, long len);
int check_equal_upper(const unsigned char* text, long len);
void swap_latin_case(unsigned char* text, long len);
}

// Результат разбора введённой строки.
enum Status { OK, NO_DOT, EMPTY, TOO_LONG, BAD_CHAR };

// ---------- Перекодировка ----------
// Терминал Mac работает в UTF-8, а процедуры рассчитаны на Windows-1251,
// где каждая буква, в том числе русская, занимает ровно один байт.
bool convert(const string& in, string& out, const char* to, const char* from) {
    iconv_t cd = iconv_open(to, from);
    if (cd == (iconv_t)-1) return false;

    out.assign(in.size() * 4 + 4, '\0');
    char* src = const_cast<char*>(in.data());
    size_t srcLeft = in.size();
    char* dst = &out[0];
    size_t dstLeft = out.size();

    size_t result = iconv(cd, &src, &srcLeft, &dst, &dstLeft);
    iconv_close(cd);
    if (result == (size_t)-1) return false;

    out.resize(out.size() - dstLeft);
```

```
    return true;
}

// ---------- Часть 1. Ввод ----------
// Берёт текст до первой точки и переводит его в Windows-1251.
Status parseInput(const string& line, string& text1251) {
    size_t dot = line.find('.');
    if (dot == string::npos) return NO_DOT;

    string text = line.substr(0, dot);
    if (text.empty()) return EMPTY;

    if (!convert(text, text1251, "WINDOWS-1251", "UTF-8")) return BAD_CHAR;
    if (text1251.size() > 100) return TOO_LONG;
    return OK;
}

// ---------- Часть 2. Проверка условия ----------
bool conditionHolds(string& text1251) {
    const unsigned char* p = (const unsigned char*)text1251.data();
    return check_equal_upper(p, (long)text1251.size()) == 1;
}

// ---------- Часть 3. Преобразование ----------
void transformText(string& text1251) {
    unsigned char* p = (unsigned char*)&text1251[0];
    swap_latin_case(p, (long)text1251.size());
}

// ---------- Часть 4. Вывод ----------
void printText(const string& label, const string& text1251) {
    string utf8;
    convert(text1251, utf8, "UTF-8", "WINDOWS-1251");
    cout << label << utf8 << "\n";
}

// Все четыре части подряд для одной строки; нужна для самопроверки.
// Возвращает статус; при OK в condition и result лежит итог.
Status runAll(const string& line, bool& condition, string& result) {
    string text1251;
    Status st = parseInput(line, text1251);
    if (st != OK) return st;

    if (text_is_valid((const unsigned char*)text1251.data(), (long)text1251.size()) == 0) return BAD_CHAR;

    condition = conditionHolds(text1251);
    if (condition) transformText(text1251);
    convert(text1251, result, "UTF-8", "WINDOWS-1251");
    return OK;
}

// ---------- Самопроверка ----------
// Эталон написан на C++ независимо от ассемблера и работает с UTF-32 буквами.
bool refCondition(const u32string& t) {
    int lat = 0, rus = 0;
    for (char32_t c : t) {
        if (c >= U'A' && c <= U'Z') lat++;
        if ((c >= U'А' && c <= U'Я') || c == U'Ё') rus++;
    }
    return lat == rus;
}

u32string refTransform(u32string t) {
    for (char32_t& c : t) {
        if (c >= U'A' && c <= U'Z') c = c - U'A' + U'a';
        else if (c >= U'a' && c <= U'z') c = c - U'a' + U'A';
```

```
    }
    return t;
}

string toUtf8(const u32string& t) {
    string out;
    for (char32_t c : t) {
        if (c < 0x80) out += (char)c;
        else if (c < 0x800) { out += (char)(0xC0 | (c >> 6)); out += (char)(0x80 | (c & 0x3F)); }
        else { out += (char)(0xE0 | (c >> 12)); out += (char)(0x80 | ((c >> 6) & 0x3F)); out += (char)(0x80 | (c & 0x3F)); }
    }
    return out;
}

int runTests() {
    int failed = 0, total = 0;

    // Контрольные случаи: строка, ожидаемое условие, ожидаемый результат.
    struct Case { const char* line; bool cond; const char* result; };
    Case cases[] = {
        {"АаБб.", true, "аАБб"},
        {"АВВГ.", true, "авВГ"},
        {"АВСА.", false, "АВСА"},
        {"АЁ.", true, "аЁ"},
        {"abc 123.", true, "АВС 123"},       // букв обоих видов нет: 0 = 0
        {"привет.", true, "привет"},         // строчные русские не считаются
        {"Hello Мир.", true, "hELLO Мир"},    // Н и М: по одной прописной
        {"Я.", false, "Я"},
        {"Q.", false, "Q"},
        {"QЯ. хвост после точки", true, "qЯ"}, // всё после первой точки игнорируется
    };

    for (const Case& c : cases) {
        total++;
        bool cond = false;
        string result;
        Status st = runAll(c.line, cond, result);
        if (st != OK || cond != c.cond || result != c.result) {
            failed++;
            cout << "ОШИБКА на \"" << c.line << "\": получено условие " << cond << ", результат \"" << result << "\"\n";
        }
    }

    // Ошибочный ввод.
    struct Bad { const char* line; Status st; };
    Bad bad[] = {{"без точки", NO_DOT}, {".", EMPTY}, {"символ ☺.", BAD_CHAR}, {"таб\tтаб.", BAD_CHAR}};
    for (const Bad& b : bad) {
        total++;
        bool cond = false;
        string result;
        if (runAll(b.line, cond, result) != b.st) {
            failed++;
            cout << "ОШИБКА: неверный статус для \"" << b.line << "\"\n";
        }
    }
    // Граница длины: 101 символ - ошибка, 100 символов - допустимо.
    bool cond = false;
    string result;
    total++;
    if (runAll(string(101, 'a') + ".", cond, result) != TOO_LONG) { failed++; cout << "ОШИБКА: 101 символ\n"; }
    total++;
    if (runAll(string(100, 'a') + ".", cond, result) != OK) { failed++; cout << "ОШИБКА: 100 символов\n"; }

    // Случайные тексты. Алфавит подобран так, чтобы условие часто выполнялось.
    const u32string alphabet = U"ABCZabcz АБВЯЁабвёя0123,;!";
    srand(2026);
```

```
    for (int k = 0; k < 5000; k++) {
        int len = 1 + rand() % 100;
        u32string t;
        for (int i = 0; i < len; i++) t += alphabet[rand() % alphabet.size()];
        total++;
        Status st = runAll(toUtf8(t) + ".", cond, result);
        bool wantCond = refCondition(t);
        string want = toUtf8(wantCond ? refTransform(t) : t);
        if (st != OK || cond != wantCond || result != want) {
            failed++;
            cout << "ОШИБКА на случайном тексте: " << toUtf8(t) << "\n";
        }
    }

    cout << "Проверено случаев: " << total << ", ошибок: " << failed << "\n";
    return failed == 0 ? 0 : 1;
}

// ---------- Главная функция ----------
int main(int argc, char* argv[]) {
    if (argc > 1 && string(argv[1]) == "--test") return runTests();

    cout << "Лабораторная работа №2, вариант 6\n";
    cout << "Условие: прописных латинских букв столько же, сколько прописных русских (Ё входит в русские).\n";
    cout << "Если условие выполнено, у каждой латинской буквы меняется регистр.\n";
    cout << "Введите текст до 100 символов и в конце поставьте точку: ";

    // Часть 1. Ввод
    string line;
    getline(cin, line);
    string text1251;
    Status st = parseInput(line, text1251);
    if (st == NO_DOT)   { cout << "Ошибка: в конце текста нет точки.\n"; return 1; }
    if (st == EMPTY)    { cout << "Ошибка: текст пустой.\n"; return 1; }
    if (st == TOO_LONG) { cout << "Ошибка: в тексте больше 100 символов.\n"; return 1; }
    if (st == BAD_CHAR) { cout << "Ошибка: есть символ, которого нет в Windows-1251.\n"; return 1; }
    if (text_is_valid((const unsigned char*)text1251.data(), (long)text1251.size()) == 0) {
        cout << "Введённая последовательность не является текстом (есть управляющие символы), преобразование не
выполняется.\n";
        return 1;
    }
    printText("Исходный текст: ", text1251);

    // Часть 2. Проверка условия
    bool ok = conditionHolds(text1251);
    cout << "Условие (поровну прописных латинских и русских букв): " << (ok ? "ВЫПОЛНЕНО" : "НЕ ВЫПОЛНЕНО") << "\n";

    // Часть 3. Преобразование
    if (ok) {
        transformText(text1251);
    } else {
        cout << "Преобразование не выполняется.\n";
    }

    // Часть 4. Вывод
    printText("Результат: ", text1251);
    return 0;
}
```

Результаты работы

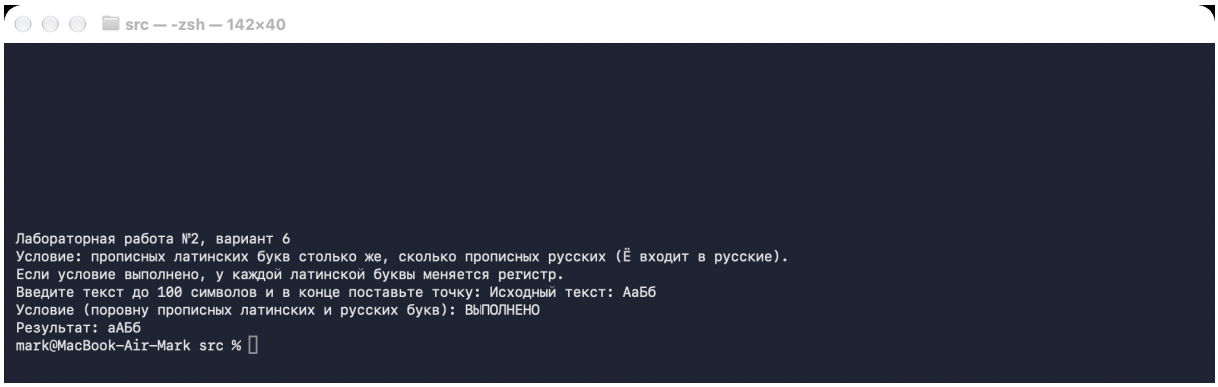


Рисунок 1 – Условие выполнено: текст «АаБб» → «аАБб»

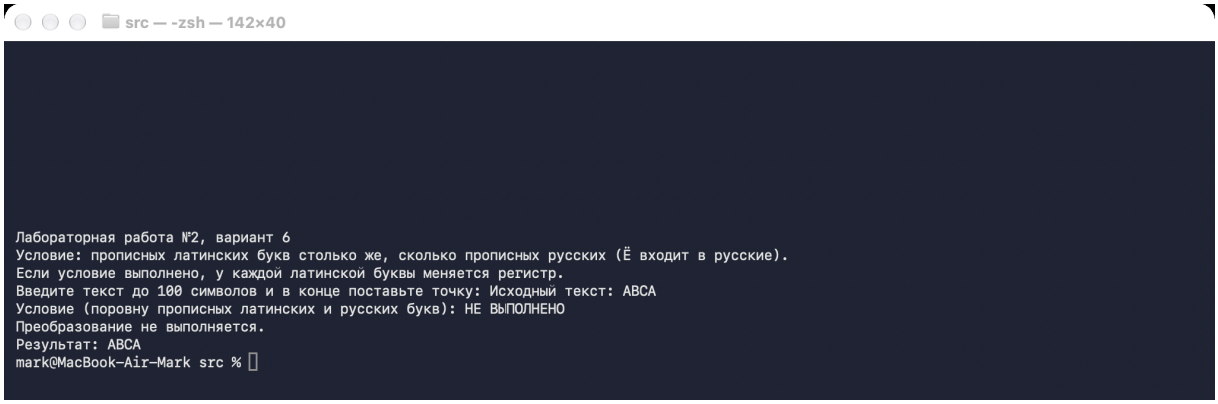


Рисунок 2 – Условие не выполнено: текст «ABCA» не меняется

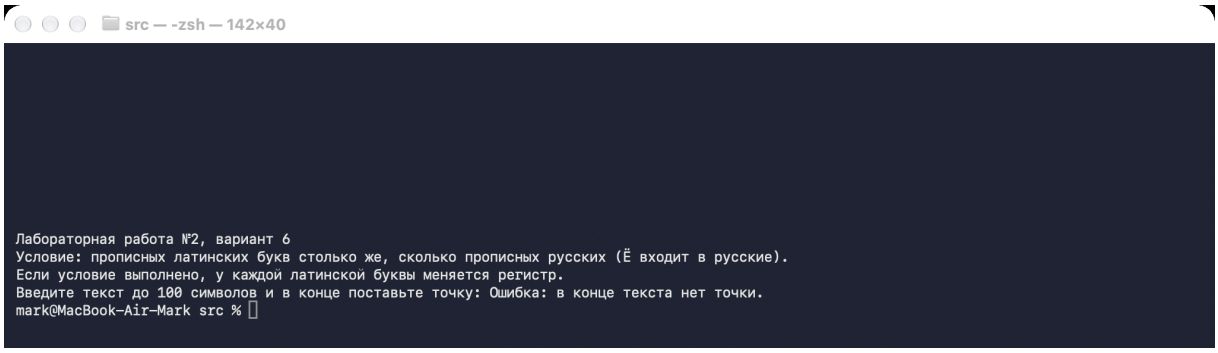


Рисунок 3 – Ошибочный ввод: нет точки в конце текста



Рисунок 4 – Автоматическая проверка: 5016 случаев, ошибок нет

Проверка. Ассемблерные процедуры сравнивались с независимой реализацией на C++: 16 контрольных случаев (включая границы длины 100 и 101 символ, ввод без точки, пустой текст, символ вне Windows-1251, управляющий символ) и 5000 случайных текстов – ошибок нет. Для проверки самого теста в ассемблерный код

намеренно внесена ошибка, и тест показал 523 ошибки. Запуск под ASan/UBSan замечаний не выдал.

Вывод: изучил программную модель микропроцессора и работу с регистровой и оперативной памятью; написал на ассемблере процедуры проверки условия и преобразования текста по варианту 6 и проверил их работу.